

FORMATION EMBARQUÉE

TP5 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 3 : ADC + DMA circulaire et console (2 h)

En-tête pédagogique

Objectifs	Configurer un ADC multi-canaux en DMA circulaire ; comprendre le transfert sans CPU ; réutiliser le ring buffer pour une console UART
Prérequis	Séances 1-2 ; module 10 §4.4/§4.6 ; TD 01 (ring buffer)
Outils	CubeIDE ; potentiomètre sur PA0
Livrable	Tableau ADC rempli en tâche de fond + console à commandes

Déroulé minuté

0:00-0:40 — ADC + DMA circulaire

Le motif star du STM32 : l'ADC scanne plusieurs canaux et **remplit un tableau tout seul**, sans le CPU, en boucle. Configuration CubeMX : 1. ADC1 : activer 2 canaux (IN0 = PA0 potentiomètre, IN16 = capteur de température interne), **Scan Conversion Mode = Enabled**, **Continuous Conversion = Enabled**. 2. DMA : ajouter une requête ADC1, **mode Circular**, direction Peripheral → Memory, données Half Word (16 bits). 3. Régénérer.

```
/* USER CODE BEGIN PV */
volatile uint16_t adc[2];           // rempli par le DMA en tâche de fond
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t *)adc, 2); // et... c'est tout !
/* USER CODE END 2 */
```

Après ce `Start_DMA`, `adc[0]` et `adc[1]` se mettent à jour en permanence sans une seule ligne dans la boucle. C'est le DMA (Direct Memory Access) qui copie périphérique → mémoire.

0:40-1:00 — Moyenne glissante et la démonstration DMA

Ajoute une moyenne glissante sur 16 échantillons de `adc[0]` (réutilise l'esprit du TD 01 / la moyenne glissante du TD 07 §2). Affiche la consigne convertie toutes les 500 ms.

✓ **Point de contrôle DMA (le plus formateur)** : pose un **point d'arrêt** dans la boucle, ouvre la vue « Live Expressions » ou la vue mémoire sur `adc`. Tourne le potentiomètre pendant que le CPU est **arrêté** au point d'arrêt : `adc[0]` **continue de changer**. Preuve visuelle que le transfert se fait sans le CPU. Savoir montrer ça = avoir compris le DMA.

1:00-1:50 — Console UART à ring buffer

Réutilise le ring buffer du TD 01 (`code/c/ring_buffer.c`) et le pattern console du TD 10 exercice 2.

Réception par interruption :

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART2) {
        rb_put(&rb_rx, octet_rx);
        HAL_UART_Receive_IT(&huart2, &octet_rx, 1); // RÉARMER (piège n°1)
    }
}
```

Commandes à implémenter : `status` (mesures + uptime + version), `log on|off`, `seuil <valeur>`. L'ISR ne fait que stocker ; la boucle vide le ring buffer, assemble les lignes (bornées !) et les traite. **Règle « ISR courte »** du module 00 en pratique.

✓ **Point de contrôle console** : `status` renvoie les mesures ADC + BME280 + uptime ; colle 500 caractères d'un coup dans le terminal → pas de crash, pas d'octets mélangés (le ring buffer absorbe).

1:50-2:00 — Commit + journal

`git commit -m "TP5 seance 3 : ADC+DMA circulaire + console ring buffer"`. Journal : décris ce que tu as observé au point d'arrêt (le tableau ADC qui bouge CPU arrêté). C'est LA preuve que le DMA travaille en parallèle.

Erreurs fréquentes

Symptôme	Cause	Remède
<code>adc[]</code> figé	DMA pas en Circular, ou Start_DMA absent	mode Circular + longueur = nb canaux
<code>adc[0]</code> et <code>adc[1]</code> inversés	ordre de rang des canaux dans CubeMX	vérifier l'ordre de scan
Console reçoit 1 seul caractère	<code>Receive_IT</code> pas réarmé dans le callback	réarmer à chaque octet
Octets mélangés en collant du texte	pas de ring buffer, parsing dans l'ISR	ISR = stocker, boucle = parser
Débordement de ligne	tampon de ligne non borné	<code>if (pos < taille-1)</code>

Travail à la maison (30 min)

Compare le capteur de température **interne** du STM32 (`adc[1]` , à convertir avec la formule du reference manual) et le BME280. Ils diffèrent (le capteur interne mesure la puce, qui chauffe). Écris dans ton journal l'écart observé et pourquoi — c'est un piège classique quand on croit mesurer l'ambiance avec le capteur interne.

➡ Fiche suivante : **Séance 4 — Architecture FreeRTOS**